

Mathematical Derivations

1 Temporal Integration

The visualization operates on a normalized timeline variable $T \in [0.0, 1.0]$. The continuous-time differential equation governing the evolution of T with respect to real-world wall-clock time t_w is:

$$\frac{dT}{dt_w} = \frac{S}{D} \quad (1)$$

where S is the unitless playback speed multiplier and D is the total physical duration (in seconds) of the animation at $S = 1$.

In the application's `requestAnimationFrame` loop, this is approximated using an explicit Euler integration scheme. Given the varying frame delta $\Delta t_w = dt$, the recursive discrete state update is:

$$T_{n+1} = T_n + \Delta t_w \left(\frac{S}{D} \right) \quad (2)$$

The implementation bounds $\Delta t_w \leq 0.1$ seconds to ensure the local truncation error of the Euler step does not cause massive discontinuities following browser thread suspension. Because the function $T(t_w)$ is strictly linear, the Euler method is exact (zero truncation error) outside of the clamping boundary.

2 Kinematic Overshoot Calculus: `easeOutBackSoft`

The blueprint proposes an `easeOutBackSoft` function to simulate mechanical gimbal momentum. The generalized cubic bezier ease-out-back function is defined over $t \in [0, 1]$ as:

$$E(t) = 1 + c_3(t - 1)^3 + c_1(t - 1)^2 \quad (3)$$

where $c_3 = c_1 + 1$.

3 Geometric Occlusion and Horizon Culling

The `useProjectedPoints` hook utilizes a dot-product test to cull labels situated on the occluded hemisphere of the spacecraft. Let $\mathbf{A}$ be the vector from the model origin to the anchor point, and $\mathbf{C}$ be the vector from the model origin to the camera.

The angle θ between these vectors satisfies:

$$\mathbf{A} \cdot \mathbf{C} = \|\mathbf{A}\| \|\mathbf{C}\| \cos \theta \implies \cos \theta = \frac{\mathbf{A} \cdot \mathbf{C}}{\|\mathbf{A}\| \|\mathbf{C}\|} \quad (4)$$

If the geometry were a perfect sphere of radius $R = \|\mathbf{A}\|$, the exact geometric horizon (the tangent boundary visible from distance $D = \|\mathbf{C}\|$) occurs at an angle θ_H where:

$$\cos(\theta_H) = \frac{R}{D} \quad (5)$$

For an orthographic projection ($D \rightarrow \infty$), the horizon angle approaches 90° ($\cos \theta_H = 0$). In perspective projection, the horizon angle is strictly less than 90° , meaning more than half the sphere is occluded.

The application utilizes a threshold of $\cos \theta > -0.35$.

$$\theta_{threshold} = \arccos(-0.35) \approx 110.48^\circ \quad (6)$$

This establishes a $\sim 20^\circ$ transition zone beyond the orthographic equator (90°). This mathematical buffer zone is strictly required to prevent labels from abruptly disappearing due to the non-spherical, faceted nature of the spacecraft procedural geometry. The interpolation of CSS opacity is mapped linearly to the domain $\cos \theta \in [-0.35, -0.15]$.

4 Attitude Kinematics: Quaternion SLERP

Euler angle interpolation suffers from gimbal lock during the Saturn Orbit Insertion retrograde burn, where pitch approaches 180° . A robust attitude control model requires unit quaternions $q = w + xi + yj + zk$.

To interpolate the orientation $q(t)$ between two keyframe quaternions q_0 and q_1 , we compute the angle Ω between them using the dot product $\cos \Omega = q_0 \cdot q_1$. The Spherical Linear Interpolation (SLERP) is:

$$q(t) = \frac{\sin((1-t)\Omega)}{\sin \Omega} q_0 + \frac{\sin(t\Omega)}{\sin \Omega} q_1 \quad (7)$$

This guarantees constant angular velocity interpolation along the shortest great circle connecting the orientations, avoiding the non-linear velocity artifacts inherent to Euler interpolation.

5 Ephemeris Continuity: Catmull-Rom Spline Basis

Linear interpolation between ephemeris keyframes produces C0 continuity, resulting in visible kinks in the trajectory. Relying on `THREE.CatmullRomCurve3` as the solution could work, and I will experiment with it, but deriving the explicit basis matrix allows for deterministic evaluation without framework overhead.

Given four control points P_0, P_1, P_2, P_3 , the interpolated position $P(t)$ for $t \in [0, 1]$ between the inner two points requires the centripetal Catmull-Rom basis matrix:

$$P(t) = \frac{1}{2} \begin{bmatrix} 1 & t & t^2 & t^3 \end{bmatrix} \begin{bmatrix} 0 & 2 & 0 & 0 \\ -1 & 0 & 1 & 0 \\ 2 & -5 & 4 & -1 \\ -1 & 3 & -3 & 1 \end{bmatrix} \begin{bmatrix} P_0 \\ P_1 \\ P_2 \\ P_3 \end{bmatrix} \quad (8)$$

This matrix evaluation guarantees C1 continuity across segment boundaries, ensuring the velocity vectors match exactly at the ephemeris anchor points.

6 Planetary Scale Z-Buffer: Logarithmic Depth Transformation

Rendering Saturn as a background body with true physical scale yields a near/far ratio of $\sim 6,000,000 : 1$. A standard 24-bit linear depth buffer collapses at this scale, resulting in severe Z-fighting.

This requires mapping the linear view-space z coordinates logarithmically to the $[-1, 1]$ clip space before the perspective divide. Given the camera far plane f and a precision constant C , the logarithmic depth z' is:

$$z' = \frac{\log(C \cdot z + 1)}{\log(C \cdot f + 1)} \quad (9)$$

This redistributes floating-point precision, prioritizing near-field geometry (spacecraft meshes) while preventing the total loss of precision for distant macroscopic bodies.

7 Disintegration Physics: Aerodynamic Heating Rate

The Grand Finale disintegration requires a physical model mapping the normalized timeline T to the `uBurnAmount` shader uniform. An arbitrary quadratic scaling curve is insufficient.

Aerodynamic heating q is proportional to atmospheric density ρ and the cube of the velocity v :

$$q \approx \frac{1}{2} \rho v^3 \quad (10)$$

Saturn's atmospheric density increases exponentially with descending altitude h :

$$\rho(h) = \rho_0 e^{-h/H} \quad (11)$$

where H is the scale height. The accumulated structural erosion is the time integral of q . Mapping the normalized time $T \rightarrow h(T)$ allows the vertex shader to drive the ablation threshold based on integrated physical heating rather than an arbitrary easing function.

8 Huygens Trajectory: Keplerian Orbital Mechanics

The Huygens separation sequence currently utilizes a linear spatial translation. A robust trajectory requires calculating the orbital state vectors directly from Keplerian elements.

We isolate the eccentric anomaly E given the mean anomaly M and eccentricity e by solving Kepler's equation via the Newton-Raphson method:

$$M = E - e \sin E \quad (12)$$

Once E converges, we determine the true anomaly and compute the Cartesian spatial coordinates (x, y, z) to accurately plot the ballistic descent arc toward Titan, fully decoupling the probe's trajectory from Cassini's local coordinate space.